LECTURE 20

Introduction to SMT and SyGuS

The constraint-based verification techniques studied so far relied on our ability to establish the validity of logical formulas that not only include boolean structure, but also integer constraints. Moreover, as we attempt to use those techniques to reason about more complex programs, we need to reason about logical formulas that also include arrays, strings, as well as other recursive data-structures. Part of the popularity of constraint-based verification and synthesis techniques stems from the fact that in recent years, a class of solvers called Satisfiability Modulo Theory Solvers (SMT solvers for short) has become increasingly powerful and able to establish the satisfiability or unsatisfiability of logical formulas that include these combinations of constraints.

20.1 One Theory with simple boolean structure

A theory in this context corresponds to a series of axioms that give meaning to a series of predicates and functions. An algorithm for deciding whether a formula in a given theory is valid is called a *decision procedure*. The goal in Satisfiability Modulo Theories is to either find satisfying assignments or prove the unsatisfiability of a formula in first order logic that includes predicates and functions in a theory. For example, consider the language of formulas shown below:

Goals	$G :=$	$L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x.G$
Hypotheses	$H :=$	$L \mid true \mid H_1 \wedge H_2$
Literals	$L :=$	$p(E_1, \ldots, E_k)$
Expressions	$E :=$	$x \mid f(E_1, \ldots E_m)$

Meaning of a formula

$\models G$ means that formula G holds

- $\models true$
- $\models G_1 \wedge G_2$ when $\models G_1$ *and* $\models G_2$
- $\models \forall x.\, G$ when for all $n \in D \models G[n/x]$ where D is the domain of x
- $\models H \Rightarrow G$ when $\models G$ whenever $\models H$
- $\models p(E_1, \ldots E_k)$ when p is true according to the theory

The literals and expressions in the language are defined in terms of the predicates and functions in the theory. The goal is to determine whether a formula is valid for all values of x in a given domain. The figure shows what it means for a formula in this theory to be valid. Most of the definitions are straightforward. The last one basically says that a predicate is valid if it evaluates to true according to the semantics of the predicate defined by the theory.

Now, suppose we want to prove that a formula holds. We can define a procedure $Prove(G) := Prove(true, G)$, where $Prove(H, G)$ decides whether goal G can be proved under hypothesis H. The procedure can be defined recursively as follows:

$$
\begin{array}{ll}
Prove(H, true) & = true \\
Prove(H, G_1 \wedge G_2) & = Prove(H, G_1) \wedge Prove(H, G_2) \\
Prove(H, H_1 \Longrightarrow G) & = Prove(H \wedge H_1, G) \\
Prove(H, \forall x.G) & = Prove(H, G[a/x]) \text{where } a \text{ is a fresh variable}
\end{array}
$$

How about $Prove(H, L)$? In this case, we have to prove a predicate in the theory assuming a conjunction of other predicates in the theory.

$$
\begin{array}{c}
\forall x.(L_0 \wedge \ldots L_k \Longrightarrow L) \\
\text{or equivalently} \\
\neg(\exists x.L_0 \wedge \ldots L_k \wedge \neg L)
\end{array}
$$

In other words, $Prove(H, L) = UNSAT(H \wedge \neg L)$. So what this means is that proving the validity of any formula in the language of Goals above reduces to proving unsatisfiability of conjunctions of predicates in the theory. How to do that, depends on the specifics of the theory in question. We now discuss a few popular theories.

20.2 Linear integer arithmetic

Example: Linear Inequalities

Goals:	$G \coloneqq L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x.G$
Hypotheses	$H \coloneqq L \mid true \mid H_1 \wedge H_2$
Literals	$L \coloneqq p(E_1, \ldots, E_k)$
Expressions	$E \coloneqq n \mid f(E_1, \ldots, E_m)$

Expressions:	$c * x_i \mid E + E$
Literals:	$E \geq 0 \mid \mathrm{E} = 0$

$$
\begin{array}{lll}
\forall y, y_0, k, k_0, t, y', t', (y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) & \Rightarrow & y = k_0 + t \\
& \wedge & y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\
& \wedge & y' = k_0 + t' \wedge \neg(t' > 0) \Rightarrow y' \leq k_0
\end{array}
$$

Linear Inequalities

$$Prove(true, ((y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) \Rightarrow \wedge \begin{array}{c} y = k_0 + t \\ y' = k_0 + t' \wedge (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \; y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array}))$$

$$\Downarrow$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \begin{array}{c} y = k_0 + t \\ \wedge \; y' = k_0 + t' \wedge (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \; y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array})$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \begin{array}{c} y = k_0 + t \\ \wedge \; y' = k_0 + t' \wedge (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \; y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array})$$

$$\Downarrow$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y = k_0 + t)$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \; y' = k_0 + t' \wedge (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1))$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0)$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \begin{array}{c} y = k_0 + t \\ \wedge \; y' = k_0 + t' \wedge (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \; y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array})$$

$$\Downarrow$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y = k_0 + t)$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge \; y' = k_0 + t' \wedge (t' > 0), y' - 1 = k_0 + (t' - 1))$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y' = k_0 + t' \wedge (t' \leq 0), y' \leq k_0)$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \left(\begin{array}{c} y = k_0 + t \\ \wedge \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \quad y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array} \right))$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y \neq k_0 + t)$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge \ y' = k_0 + t' \ \wedge \ t' > 0 \wedge y' - 1 \neq k_0 + (t' - 1))$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y' = k_0 + t' \wedge t' \leq 0 \wedge y' > k_0)$$

Is this enough to check VCs?

Given a definition of VC

- $VC(skip, \{Q\}) = Q$
- $VC(x = e, \{Q\}) = Q[e/x]$
- $VC(C1; C2\{Q\}) = VC(C1\{VC(C2\{Q\})\})$
- $VC(if\ B\ then\ C1\ else\ C2\{Q\}) = \big(B \Rightarrow VC(C1\{Q\})\big) \wedge (\neg B \Rightarrow VC(C2\{Q\}))$
- $VC(while_I\ e\ do\ c, B) =$
  $I \wedge \ \forall x_1, \ldots x_n\ I \Rightarrow (e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow B)$
  - Where x_i are variables modified in c.
- $VC(assert\ B\ \{Q\}) = \{Q \wedge B\}$
- $VC(assume\ B\{Q\}) = \{B \Rightarrow Q\}$

Consider the case where the domain D corresponds to the integers, and the only function available is sum (multiplication by a constant is just a shorthand for repeated sum). The predicates are >, >=, = and ≠. In this case, a conjunction of predicates in the theory corresponds to a system of linear equalities and inequalities over the integers, and there are many well known algorithms to solve such systems.

As the figure shows, the verification condition we generated for the example in Lecture 12 can be solved with the $Prove$ procedure using only the theory of linear arithmetic. Resolving the verification condition reduces to proving the unsatisfiability of a series of conjunctions of linear inequalities.

More generally, we can see that with a slight modification to the rule for `if` relative to the one shown in Lecture 12, any verification condition can be proven with $Prove$ as long as the invariants and postconditions are conjunctions of literals in the theory of integer linear arithmetic.

20.3 Equality with uninterpreted functions.

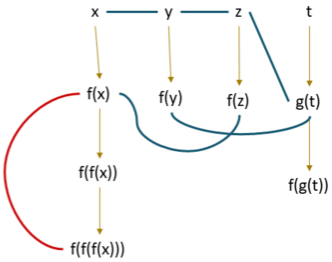
Equality with uninterpreted functions

Equality with uninterpreted functions

- Symbols: $=, \neq, f, g, \ldots$
- Axioms:
- $\frac{}{E=E} \qquad \frac{E_2=E_1}{E_1=E_2} \qquad \frac{E_1=E_2 \quad E_2=E_3}{E_1=E_3}$
- $\frac{E_1=E_2}{f(E_1)=f(E_2)}$

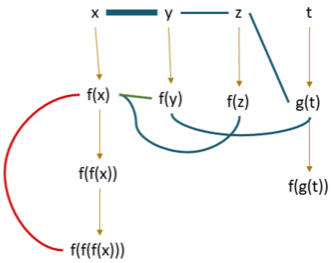
Equality with UF

x=y
y=z
f(x)=f(z)
g(t)=z
f(y) = g(t)
f(x) != f(f(f(x)))



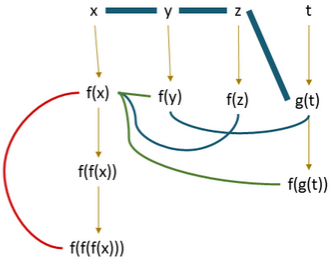
Equality with UF

x=y
y=z
f(x)=f(z)
g(t)=z
f(y) = g(t)
f(x) != f(f(f(x)))



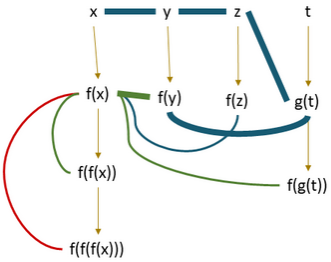
Equality with UF

x=y
y=z
f(x)=f(z)
g(t)=z
f(y) = g(t)
f(x) != f(f(f(x)))



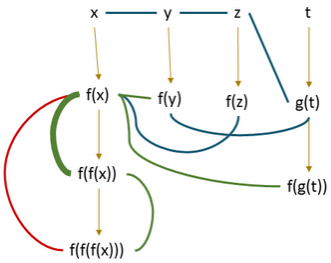
Equality with UF

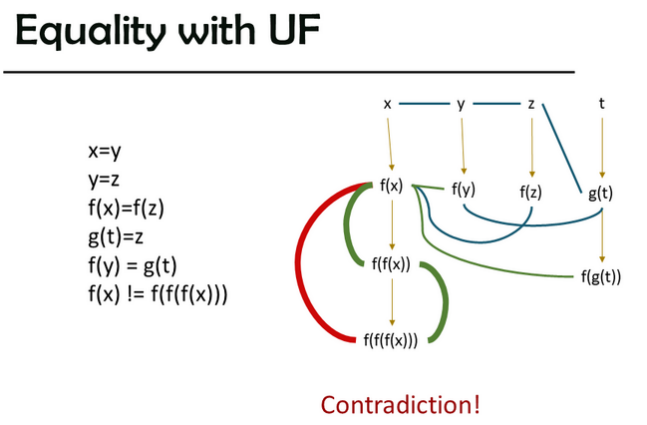
x=y
y=z
f(x)=f(z)
g(t)=z
f(y) = g(t)
f(x) != f(f(f(x)))



Equality with UF

x=y
y=z
f(x)=f(z)
g(t)=z
f(y) = g(t)
f(x) != f(f(f(x)))

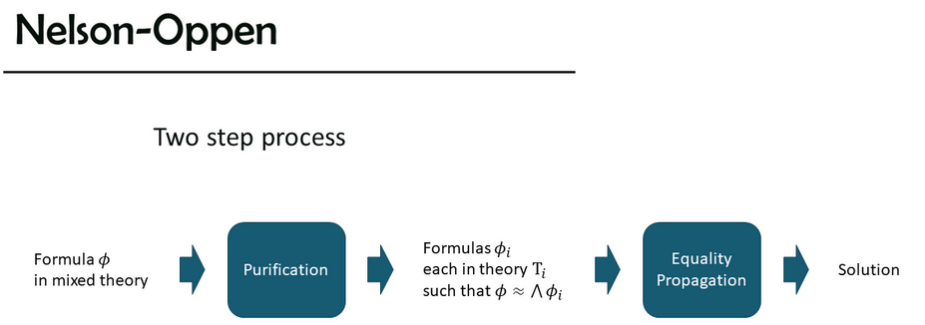




Another widely used theory is the theory of equality with uninterpreted functions. We already talked about uninterpreted functions in Lecture 13. This theory is often refered as the *empty theory* because it does not impose any additional semantics on the function symbols and the only available predicate is the equality predicate. The equality relation is defined by the standard axioms of reflexivity, commutativity and transitivity, and there is an additional axiom of *congruence* which states that if the inputs to a function are equal, then its output is equal as well. The figure illustrates the basic idea of the decision procedure for this theory; the basic idea is to keep track of all the equivalence classes of values and propagate equalities based on both transitivity and congruence. If at any point, one of the equalities contradicts an inequality, the formula is clearly unsatisfiable. If we manage to get the closure of all the transitivities and closures without reaching a contradiction, then the formula is satisfiable.

20.4 Nelson-Oppen

So at this point, we know how to solve formulas that involve a single theory and a simple boolean structure. What happens now if we have a formula that combines multiple theories? For example, consider the formula $f(x + y) \neq f(g(u) + g(v)) \wedge g(u) = y \wedge g(v) = x$.



In 1979, Greg Nelson and Derek Oppen developed a technique for combining decision procedures for different theories that under certain conditions yields a sound and complete algorithm to solve problems like the one above [1]. Before Nelson-Oppen, there had been some work on decision procedures for particular combinations of theories; in fact, Shostak published an efficient decision procedure for linear integer arithmetic and uninterpreted functions [2] only months before Nelson and Oppen. But the Nelson-Oppen procedure is quite general and easy to understand, so it will be the focus of this section.

The high-level idea for the algorithm is illustrated by the figure. The first step in the algorithm is called Purification, and it involves breaking apart a formula into a conjunction of formulas from individual theories by introducing additional variables. The idea is that for each theory T_j we want a formula ϕ_j such that the conjunction of all the ϕ_js is equisatisfiable with the original formula ϕ. To do this, we can start by assigning each $p(E_1, \ldots, E_n)$ to the formula for the theory T_i to which p belongs. Then, for every literal $p(E_1, \ldots, E_n)$ if p belongs to some theory T_i and E_k belongs to a different theory T_j, then we need to replace f_k in p with a fresh variable w and add a constraint $w = E_k$ to the formula ϕ_j for theory T_j. Similarly, for every expression $E(f_1, \ldots, f_n)$, if E belongs to some theory T_i and f_k belongs to a different theory T_j, then we need to replace f_k in E with a fresh variable w and add a constraint $w = f_k$ to the formula ϕ_j for theory T_j. This process is illustrated in the figure for the case of the running example.

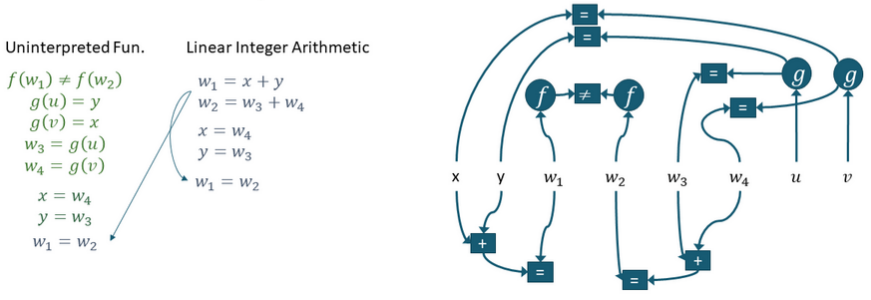
Example

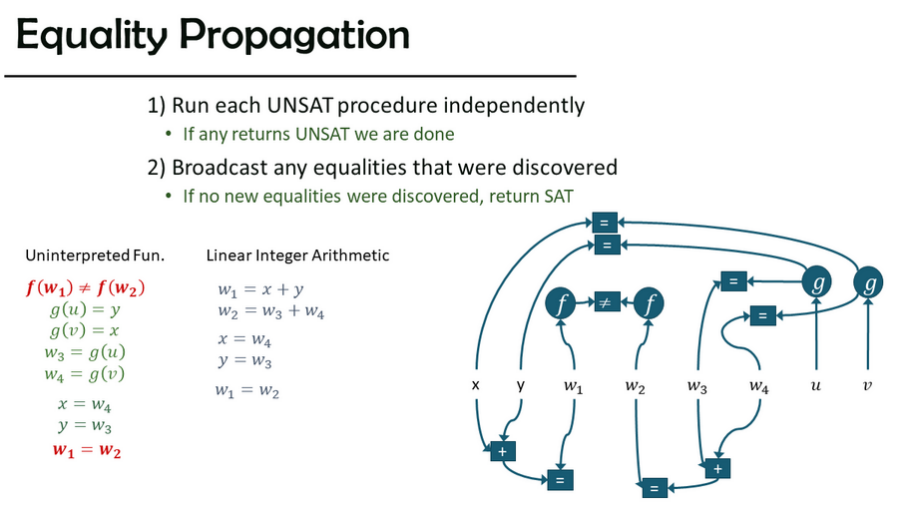
$$f(x+y) \neq f\big(g(u)+g(v)\big) \wedge g(u) = y \wedge g(v) = x$$

$f(w_1) \neq f(w_2)$
$g(u) = y$
$g(v) = x$
$w_1 = x + y$
$w_2 = w_3 + w_4$
$w_3 = g(u)$
$w_4 = g(v)$

Equality Propagation

1) Run each UNSAT procedure independently
   - If any returns UNSAT we are done
2) Broadcast any equalities that were discovered
   - If no new equalities were discovered, return SAT





The second step in the Nelson-Oppen algorithm is *equality propagation*. This is an iterative process that alternates between two phases:

- Run the UNSAT procedure for each formula ϕ_i independently; if any returns UNSAT we are done as the whole formula is clearly unsat.
- Broadcast any equalities among shared variables that were discovered as part of the process. If no new equalities were discovered, return SAT. If additional equalities were discovered, repeat the process starting from the previous step.

So the process will keep running until either one of the theory solvers returns UNSAT, or no new equalities are discovered. This procedure is clearly sound; when one of the decision procedures says UNSAT, the problem is clearly unsatisfiable. In general, the procedure is only complete (meaning it always says UNSAT when required) under a *convexity condition*.

Convex theories

Consider the formula:

$$1 \leq x$$
$$x \leq 2$$
$$f(x) \neq f(1)$$
$$f(x) \neq f(2)$$

Theory E
$1 \leq x$
$x \leq 2$
$w_1 = 1$
$w_2 = 2$

Theory R
$f(x) \neq f(w_1)$
$f(x) \neq f(w_2)$

- No equalities to propagate
- The two subformulas are satisfiable
- Yet, the problem is clearly unsat!

Backtracking

Consider the formula:

$$1 \leq x$$
$$x \leq 2$$
$$f(x) \neq f(1)$$
$$f(x) \neq f(2)$$

ϕ_E in theory E
$1 \leq x$
$x \leq 2$
$w_1 = 1$
$w_2 = 2$

ϕ_R in theory R
$f(x) \neq f(w_1)$
$f(x) \neq f(w_2)$

- Formula from theory E implies $(x = w_1) \vee (x = w_2)$.
- Try $\phi_R \wedge (x = w_1)$
- Try $\phi_R \wedge (x = w_2)$
- Since both are unsat, we can say UNSAT

A theory is convex if for every A theory is convex if for every conjunctive formula F,

$$F \Rightarrow \wedge_i x_i = y_i$$
implies
$$\exists i.F \Rightarrow x_i = y_i$$

To illustrate what this means, consider the problem from the figure. The subproblems from both formulas are both satisfiable, and there are clearly no equalities to propagate, yet it is clear that the combination of both is unsatisfiable. The problem is that the theory on the left (Theory E) is not convex. The formula on the left clearly implies that $(x = 1) \vee (x = 2)$, but it does not imply

either of those equalities individually. Therefore, there is no single equality that can be propagated.

A way to avoid this problem is to introduce backtracking search. For example, as the figure illustrates, in the case of the example, both possibilities ($x = 1$ or $x = 2$) lead to unsatisfiability, which allows us to conclude that the problem is unsatisfiable.

20.5 General SMT and DPLL(T)

For problems with more arbitrary boolean structure, it is essential to be able to leverage the power of modern SAT solvers discussed in Lecture 9. There are two general approaches for leveraging SAT in the context of these problems: the *eager* approach and the *lazy* approach. To understand both of these approaches, we first need to introduce the concept of a *boolean abstraction* of a SAT problem. A boolean abstraction of a formula F is a boolean formula B_F such that $UNSAT(B_F) \Rightarrow UNSAT(F)$.

As an example, consider the following formula:

$$\begin{array}{ll} & f(x) \neq f(y) \\ \wedge & x = a \vee x = b \\ \wedge & y = a \vee y = b \\ \wedge & a = b \end{array}$$

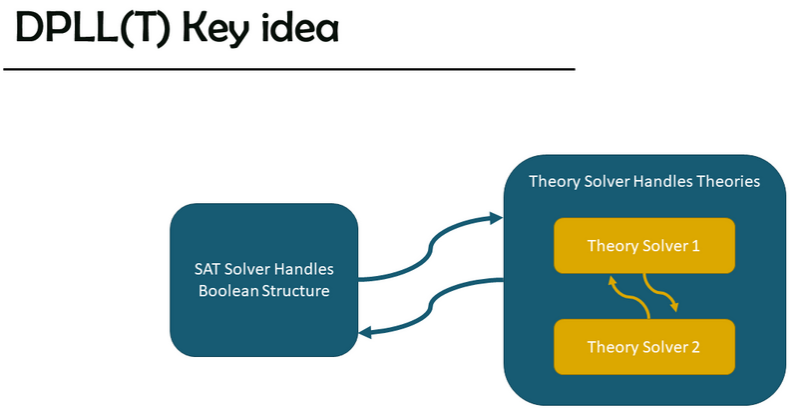
One way to construct a boolean abstraction of a formula like this is to take all the predicates in the theory and replace them with boolean variables. For example, in the example above we can replace each predicate with a variable, for example, $t_1 \equiv f(x) = f(y)$. The result is the boolean abstraction below; this is also referred sometimes as the *boolean skeleton* of the formula. Clearly, if the boolean skeleton is unsatisfiable, then the formula itself would also be unsatisfiable.

$$\begin{array}{ll} & \neg t_1 \\ \wedge & t_2 \vee t_3 \\ \wedge & t_4 \vee t_5 \\ \wedge & t_6 \end{array}$$

In the Eager approach to SMT, the goal is to construct a boolean abstraction that is equisatisfiable with the original formula, i.e. $UNSAT(F) \Rightarrow UNSAT(B_F)$. One way to do this is to explicitly generate boolean constraints corresponding to all the axioms in the theory. For example, in the example above, we can introduce an extra variable $t_8 \equiv x = y$. Then, we can add constraints corresponding to all the axioms from the theory of uninterpreted functions. For example, from congruence, we get that $t_8 \Rightarrow t_1$, and from transitivity we get all the following implications:

$$t_2 \wedge t_4 \Rightarrow t_8 \qquad t_3 \wedge t_5 \Rightarrow t_8$$
$$t_2 \wedge t_6 \Rightarrow t_3 \qquad t_4 \wedge t_6 \Rightarrow t_5$$
$$t_3 \wedge t_6 \Rightarrow t_2 \qquad t_5 \wedge t_6 \Rightarrow t_4$$

We can define a boolean abstraction by adding these additional constraints to the boolean skeleton, and the resulting formula will now be unsatisfiable like the original constraint.



This approach can be quite effective for some theories, such as the theory of bit-vectors, but in general, the main problem with this approach is that the number of boolean constraints we need to add to fully represent the theory may be prohibitively large. Instead, the idea behind the *lazy* approach is to have a theory solver interact with the SAT solver to incrementally add constraints to the boolean abstraction. The DPLL(T) approach is a framework for structuring this kind of interaction between the SAT solver and the theory solver [3].

The high-level idea is that the SAT solver starts with the boolean skeleton of the problem and proposes assignments to all the variables. The Theory solver then tries to solve a problem that is consistent with these assignments. If it succeeds, then the problem is satisfiable and we are done. If it fails, on the other hand, the theory solver must now generate a conflict clause that summarizes why that assignment did not work, and that clause can then be added to the SAT problem to force the SAT solver to produce a new assignment.

For the running example, suppose the SAT solver produces the following assignment $[t_1 = F, t_2 = T, t_3 = F, t_4 = T, t_5 = F, t_6 = T]$. The theory solver could immediately see that this assignment is not consistent with the theory, so in principle, it could return a conflict clause of the form $t_1 \wedge \bar{t_2} \wedge t_3 \wedge \bar{t_4} \wedge t_5 \wedge \bar{t_6}$ to ensure such a bad assignment never appears again. Eventually, after many attempts, the SAT solver will have accumulated enough constraints to prove that the problem is unsatisfiable.

In order for this process to be efficient, we need two things. First, it is important for the theory solver to be able to check unsatisfiability as the SAT solver makes assignments, rather than having to wait until all the assignments have been made. For example, for the assignment above, by the time the SAT solver sets $[t_1 = F, t_2 = T, t_3 = F, t_4 = T]$, the theory solver can prove unsatisfiability, without having to wait for t_5 or t_6 to be set. So it is important for the theory solver to be able to check satisfiability incrementally as the SAT solver makes assignments. Moreover, even after the aforementioned assignment, the theory solver should be able to prove that actually, the assignments to t_1, t_2 and t_4 are enough to prove unsatisfiability, so we want it to produce a conflict clause $t_1 \wedge \bar{t_2} \wedge \bar{t_4}$ that reflects this knowledge. Such a compact conflict clause will help the SAT solver prove unsatisfiability much faster than the original one with six literals.

20.6 From SMT-LIB to SyGuS

Back in 2003, the community of researchers working on SMT solvers got together to create a standard input format that would allow different research groups to develop solvers against a common interface. The result of this effort was the SMT-LIB standard, which is supported by all major SMT solvers. SMT-LIB defines a syntax to describe SMT problems, and it defines a standarized set of theories that a solver can support.

In 2013, the program synthesis community got together to propose an extension to the SMT-LIB standard to support the specification of synthesis problems. The result was the SyGuS standard [4].

The idea behind SyGuS is to allow the user to describe a grammar for a set of unknown functions and then add constraints in SMT-LIB that specify the behavioral constraints on the synthesized functions. For example, consider the SyGuS problem below (copied from the 2.0 language standard):

```
( set-logic LIA )
( synth-fun f (( x Int ) ( y Int )) Int
    ( ( I Int ) ( Ic Int ) )
    ( ( I Int (0 1 x y
                (+ I I )
                (* Ic I ))
      )
      ( Ic Int (0 1 2 ( - 1) ( - 2)) )
    )
)
( declare-var x Int )
( declare-var y Int )
( constraint (= ( f x y ) (* 2 (+ x y ))))
( check-synth )
```

The first line, `(set-logic LIA )` specifies that this code uses the theory of linear integer arithmetic (LIA). After that comes a definition for a function f to be synthesized. This function takes two integer arguments and must be generated from a grammar with two non-terminals, `I` and `Ic`. Both produce Integer expressions. `Ic` must be one of the constants `0, 1, 2, -1, -2`, whereas `I` can be a constant `0, 1`, one of the input variables `x, y`, a sum of two `I`s, or the product of an `Ic` times an `I`.

The `(declare-var )` declarations specify universally quantified variables, so the constraint specifies that $f(x, y)$ should equal $2 * (x + y)$.

Because SyGuS solvers are fully integrated with SMT solvers, they can provide strong guarantees over the results they generate (unlike sketch which is limited to bounded guarantees). If you are looking for a state-of-the-art SMT solver with SyGuS support, your best bet in 2022 is CVC5, led by Clark Barrett and Cesare Tinelli. It has good support for a number of theories, and incorporates state-of-the-art synthesis algorithms.

References

[1] Greg Nelson and Derek C. Oppen. Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst.*, 1(2):245–257, 1979. doi: 10.1145/357073.357079. URL http://doi.acm.org/10.1145/357073.357079.

[2] Robert E. Shostak. A practical decision procedure for arithmetic with function symbols. *J. ACM*, 26(2):351–360, April 1979. ISSN 0004-5411. doi: 10.1145/322123.322137. URL http://doi.acm.org/10.1145/322123.322137.

[3] Harald Ganzinger, George Hagen, Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. DPLL(T): fast decision procedures. In *Computer Aided Verification, 16th International Conference, CAV 2004, Boston, MA, USA, July 13-17, 2004, Proceedings*, pages 175–188, 2004. doi: 10.1007/978-3-540-27813-9_14. URL https://doi.org/10.1007/978-3-540-27813-9_14.

[4] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design, FMCAD 2013, Portland, OR, USA, October 20-23, 2013*, pages 1–8, 2013. URL http://ieeexplore.ieee.org/document/6679385/.